

Optimization for Data Science

From Convex Optimization to Evolutionary Algorithms

Diogo Ribeiro

ESMAD - Escola Superior de Média Arte e Design

Lead Data Scientist, Mysense.ai

✉ dfr@esmad.ipp.pt ⓘ 0009-0001-2022-7072

May 22, 2026

Outline

- 1 Introduction to Optimization
- 2 Convex Optimization
- 3 Gradient Descent Methods
- 4 Constrained Optimization
- 5 Evolutionary and Metaheuristic Algorithms
- 6 Bayesian Optimization
- 7 Practical Optimization in Data Science
- 8 Advanced Topics
- 9 Conclusion

Introduction to Optimization

What is Optimization?

Optimization Problem

Finding the best solution from a set of feasible alternatives:

$$\min_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x})$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is the objective function and $\mathcal{X}$ is the feasible set.

Components:

- **Decision variables:** $\mathbf{x} = (x_1, \dots, x_n)$
- **Objective function:** $f(\mathbf{x})$ (cost, loss, error)
- **Constraints:** $g_i(\mathbf{x}) \leq 0$, $h_j(\mathbf{x}) = 0$

Goal: Find $\mathbf{x}^*$ that minimizes (or maximizes) $f(\mathbf{x})$.

Optimization in Data Science

Machine Learning:

- Training models: minimize loss function
- Linear regression: minimize $\sum_i (y_i - \mathbf{w}^T \mathbf{x}_i)^2$
- Logistic regression: minimize cross-entropy
- Neural networks: backpropagation + gradient descent

Statistics:

- Maximum likelihood estimation
- Bayesian inference (MAP estimation)
- Robust estimation

Operations Research:

- Resource allocation
- Scheduling and routing
- Portfolio optimization

Hyperparameter Tuning:

Types of Optimization Problems

By Variable Type:

- **Continuous:** $\mathbf{x} \in \mathbb{R}^n$
- **Discrete:** $\mathbf{x} \in \mathbb{Z}^n$
- **Mixed-integer:** Both types

By Constraints:

- **Unconstrained:** $\mathcal{X} = \mathbb{R}^n$
- **Constrained:** $\mathcal{X} \subset \mathbb{R}^n$

By Objective:

- **Linear:** $f(\mathbf{x}) = \mathbf{c}^T \mathbf{x}$
- **Quadratic:** $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{c}^T \mathbf{x}$
- **Convex:** f is convex
- **Non-convex:** General case

By Number of Objectives:

- **Single-objective**
- **Multi-objective**

Complexity

Convex problems are "easy" (polynomial time). Non-convex and discrete problems are generally NP-hard.

Convex Optimization

Convex Sets and Functions

Convex Set

A set $\mathcal{C}$ is convex if for any $\mathbf{x}, \mathbf{y} \in \mathcal{C}$ and $\theta \in [0, 1]$:

$$\theta \mathbf{x} + (1 - \theta) \mathbf{y} \in \mathcal{C}$$

Convex Function

A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex if for any $\mathbf{x}, \mathbf{y}$ and $\theta \in [0, 1]$:

$$f(\theta \mathbf{x} + (1 - \theta) \mathbf{y}) \leq \theta f(\mathbf{x}) + (1 - \theta) f(\mathbf{y})$$

Equivalent conditions (differentiable f):

- **First-order:** $f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^T (\mathbf{y} - \mathbf{x})$
- **Second-order:** $\nabla^2 f(\mathbf{x}) \succeq 0$ (Hessian is positive semidefinite)

Properties of Convex Functions

Key properties:

- **Local minimum is global minimum**
- **Sublevel sets are convex:** $\{\mathbf{x} : f(\mathbf{x}) \leq \alpha\}$
- **Closed under positive combinations:** If f_1, f_2 convex, then $\alpha f_1 + \beta f_2$ convex for $\alpha, \beta \geq 0$
- **Composition rules:** $f(g(\mathbf{x}))$ may be convex under certain conditions

Common convex functions:

- Linear: $\mathbf{a}^T \mathbf{x} + b$
- Quadratic (with $\mathbf{Q} \succeq 0$): $\mathbf{x}^T \mathbf{Q} \mathbf{x}$
- Norms: $\|\mathbf{x}\|_p$ for $p \geq 1$
- Exponential: e^{ax}
- Logarithm: $-\log(x)$ on $\mathbb{R}_{++}$
- Maximum: $\max\{x_1, \dots, x_n\}$

Fundamental Property

Optimality Conditions

First-Order Optimality (Unconstrained)

For differentiable f , $\mathbf{x}^*$ is a minimum if and only if:

$$\nabla f(\mathbf{x}^*) = \mathbf{0}$$

Second-Order Optimality

Necessary: If $\mathbf{x}^*$ is a local minimum, then:

$$\nabla f(\mathbf{x}^*) = \mathbf{0} \quad \text{and} \quad \nabla^2 f(\mathbf{x}^*) \succeq 0$$

Sufficient: If $\nabla f(\mathbf{x}^*) = \mathbf{0}$ and $\nabla^2 f(\mathbf{x}^*) \succ 0$ (positive definite), then $\mathbf{x}^*$ is a strict local minimum.

Convex Optimization Problem

Standard Form

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m \\ & \mathbf{Ax} = \mathbf{b} \end{aligned}$$

where f, g_i are convex functions.

Special cases:

- **Linear Programming (LP):** f and g_i are affine
- **Quadratic Programming (QP):** f quadratic, g_i affine
- **Second-Order Cone Programming (SOCP):** Generalization of QP
- **Semidefinite Programming (SDP):** Matrix optimization

Gradient Descent Methods

Gradient Descent

Idea: Iteratively move in the direction of steepest descent.

Gradient Descent Algorithm

Starting from $\mathbf{x}_0$, iterate:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k)$$

where $\alpha_k > 0$ is the step size (learning rate).

Convergence conditions:

- For convex, L -Lipschitz continuous ∇f : converges with $\alpha_k = \frac{1}{L}$
- Convergence rate: $f(\mathbf{x}_k) - f(\mathbf{x}^*) = O(1/k)$

Step size selection:

- **Constant:** $\alpha_k = \alpha$
- **Decreasing:** $\alpha_k = \frac{\alpha_0}{k}$ or $\alpha_k = \frac{\alpha_0}{\sqrt{k}}$

Batch vs Stochastic Gradient Descent

Batch Gradient Descent:

- Use full dataset to compute gradient
- For $f(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, \mathbf{w}^T \mathbf{x}_i)$:

$$\nabla f(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \nabla \ell(y_i, \mathbf{w}^T \mathbf{x}_i)$$

- Exact gradient, but expensive for large n

Stochastic Gradient Descent (SGD):

- Use single random sample to estimate gradient

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \alpha_k \nabla \ell(y_i, \mathbf{w}_k^T \mathbf{x}_i)$$

- Fast updates, but noisy gradient estimates

Mini-batch SGD:

- Compromise: use batch of b samples
- Most common in practice (typical b : 32, 64, 128, 256)

Momentum Methods

Problem: Standard GD can be slow in ill-conditioned problems.

Gradient Descent with Momentum

$$\mathbf{v}_{k+1} = \beta \mathbf{v}_k - \alpha \nabla f(\mathbf{x}_k)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{v}_{k+1}$$

where $\beta \in [0, 1)$ is the momentum coefficient (typically 0.9).

Intuition:

- Accumulate velocity in directions of consistent gradient
- Dampens oscillations in high-curvature directions
- Accelerates progress in low-curvature directions

Nesterov Accelerated Gradient (NAG)

Adaptive Learning Rate Methods

AdaGrad (Adaptive Gradient):

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \frac{\alpha}{\sqrt{\mathbf{G}_k + \epsilon}} \odot \nabla f(\mathbf{x}_k)$$

where $\mathbf{G}_k = \sum_{i=0}^k (\nabla f(\mathbf{x}_i))^2$ (element-wise), $\epsilon \approx 10^{-8}$

- Adapts learning rate per parameter
- Good for sparse gradients
- Problem: Accumulated gradients can make learning rate too small

RMSProp:

$$\mathbf{G}_k = \beta \mathbf{G}_{k-1} + (1 - \beta)(\nabla f(\mathbf{x}_k))^2$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \frac{\alpha}{\sqrt{\mathbf{G}_k + \epsilon}} \odot \nabla f(\mathbf{x}_k)$$

- Uses exponential moving average
- Fixes AdaGrad's aggressive learning rate decay

Adam (Adaptive Moment Estimation)

Combines momentum and adaptive learning rates:

$$\mathbf{m}_k = \beta_1 \mathbf{m}_{k-1} + (1 - \beta_1) \nabla f(\mathbf{x}_k) \quad (\text{first moment})$$

$$\mathbf{v}_k = \beta_2 \mathbf{v}_{k-1} + (1 - \beta_2) (\nabla f(\mathbf{x}_k))^2 \quad (\text{second moment})$$

$$\hat{\mathbf{m}}_k = \frac{\mathbf{m}_k}{1 - \beta_1^k} \quad (\text{bias correction})$$

$$\hat{\mathbf{v}}_k = \frac{\mathbf{v}_k}{1 - \beta_2^k} \quad (\text{bias correction})$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \frac{\alpha}{\sqrt{\hat{\mathbf{v}}_k} + \epsilon} \odot \hat{\mathbf{m}}_k$$

Default hyperparameters: $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Other Modern Optimizers

AdamW:

- Decoupled weight decay from gradient-based updates
- Better regularization than Adam with L2 penalty
- Becoming preferred over Adam for many applications

RAdam (Rectified Adam):

- Addresses early training instability
- Adaptive learning rate warm-up

Lookahead:

- Meta-optimizer that can wrap any base optimizer
- Maintains slow and fast weights

Second-order methods:

- **Newton's method:** Use Hessian information
- **L-BFGS:** Limited-memory quasi-Newton
- Fast convergence but expensive per iteration

Gradient Descent Variants Comparison

Method	Momentum	Adaptive LR	Memory	Use Case
SGD	No	No	Low	Simple, well-understood
SGD+Momentum	Yes	No	Low	Standard baseline
AdaGrad	No	Yes	Medium	Sparse features
RMSProp	No	Yes	Medium	RNNs, online learning
Adam	Yes	Yes	Medium	General purpose
AdamW	Yes	Yes	Medium	With weight decay
L-BFGS	-	-	High	Small datasets

Practical Advice

- Start with Adam/AdamW with default parameters
- If overfitting, add weight decay or use AdamW
- For final tuning, try SGD with momentum and learning rate schedule
- Monitor training curves to detect issues

Constrained Optimization

Lagrangian and Duality

Primal problem:

$$\begin{array}{ll}\min_{\mathbf{x}} & f(\mathbf{x}) \\ \text{s.t.} & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \dots, p\end{array}$$

Lagrangian

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\nu}) = f(\mathbf{x}) + \sum_{i=1}^m \lambda_i g_i(\mathbf{x}) + \sum_{j=1}^p \nu_j h_j(\mathbf{x})$$

where $\boldsymbol{\lambda} \geq \mathbf{0}$ (inequality multipliers) and $\boldsymbol{\nu}$ (equality multipliers).

Dual Function

Karush-Kuhn-Tucker (KKT) Conditions

For $\mathbf{x}^*$ to be optimal (assuming constraint qualifications hold):

1. Stationarity:

$$\nabla f(\mathbf{x}^*) + \sum_{i=1}^m \lambda_i^* \nabla g_i(\mathbf{x}^*) + \sum_{j=1}^p \nu_j^* \nabla h_j(\mathbf{x}^*) = \mathbf{0}$$

2. Primal feasibility:

$$g_i(\mathbf{x}^*) \leq 0, \quad h_j(\mathbf{x}^*) = 0$$

3. Dual feasibility:

$$\lambda_i^* \geq 0$$

4. Complementary slackness:

$$\lambda_i^* (g_i(\mathbf{x}^*) + 0) = 0, \quad \forall i$$

Penalty and Barrier Methods

Penalty Methods:

- Convert constrained problem to unconstrained
- Add penalty for constraint violations:

$$\min_{\mathbf{x}} f(\mathbf{x}) + \rho \sum_{i=1}^m \max(0, g_i(\mathbf{x}))^2 + \rho \sum_{j=1}^p h_j(\mathbf{x})^2$$

- Increase penalty parameter ρ iteratively

Barrier Methods:

- Add barrier function to prevent constraint violations
- Logarithmic barrier:

$$\min_{\mathbf{x}} f(\mathbf{x}) - \frac{1}{\mu} \sum_{i=1}^m \log(-g_i(\mathbf{x}))$$

- Decrease barrier parameter μ iteratively
- Basis of interior-point methods

Projected Gradient Descent

For problems with simple constraints:

$$\min_{\mathbf{x} \in \mathcal{C}} f(\mathbf{x})$$

Projected Gradient Descent

$$\mathbf{x}_{k+1} = \Pi_{\mathcal{C}}(\mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k))$$

where $\Pi_{\mathcal{C}}(\mathbf{y}) = \arg \min_{\mathbf{x} \in \mathcal{C}} \|\mathbf{x} - \mathbf{y}\|_2$ is the projection onto $\mathcal{C}$.

Common projections:

- **Box constraints:** $\mathcal{C} = [l, u]$

$$[\Pi_{\mathcal{C}}(\mathbf{y})]_i = \max(l_i, \min(y_i, u_i))$$

- **ℓ_2 ball:** $\mathcal{C} = \{\mathbf{x} : \|\mathbf{x}\|_2 \leq r\}$

Proximal Methods

For problems with non-smooth terms:

$$\min_{\mathbf{x}} f(\mathbf{x}) + g(\mathbf{x})$$

where f is smooth, g is non-smooth but "simple" (e.g., ℓ_1 norm).

Proximal Operator

$$\text{prox}_{\alpha g}(\mathbf{y}) = \arg \min_{\mathbf{x}} \left\{ g(\mathbf{x}) + \frac{1}{2\alpha} \|\mathbf{x} - \mathbf{y}\|_2^2 \right\}$$

Proximal Gradient Method

$$\mathbf{x}_{k+1} = \text{prox}_{\alpha_k g}(\mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k))$$

Evolutionary and Metaheuristic Algorithms

Evolutionary Algorithms Overview

Inspiration: Biological evolution and natural selection.

Key concepts:

- **Population:** Set of candidate solutions
- **Fitness:** Quality of a solution (objective function value)
- **Selection:** Choose better solutions for reproduction
- **Crossover:** Combine two solutions to create offspring
- **Mutation:** Random modifications to solutions
- **Replacement:** Update population with new generation

Advantages:

- Don't require gradient information
- Can handle non-convex, multimodal, discrete problems
- Naturally handle constraints and multiple objectives
- Global search capability

Disadvantages:

Genetic Algorithm

1. **Initialize:** Random population of size N
2. **Repeat until convergence:**
 1. **Evaluate:** Compute fitness $f(\mathbf{x})$ for each individual
 2. **Selection:** Select parents based on fitness
 - Roulette wheel: probability $\propto$ fitness
 - Tournament: best of random subset
 - Rank-based: based on rank, not absolute fitness
 3. **Crossover:** Create offspring from parents
 - One-point, two-point, uniform crossover
 4. **Mutation:** Randomly modify offspring
 - Bit flip (binary), Gaussian noise (real-valued)
 5. **Replacement:** Form new population

Evolution Strategies (ES)

Designed for continuous optimization:

- Represent solutions as real vectors
- Self-adaptive mutation rates
- Strong theoretical foundations

(μ, λ) -ES

- μ parents generate λ offspring
- Select best μ from offspring (no elitism)

$(\mu + \lambda)$ -ES

- Select best μ from parents and offspring (elitism)

CMA-ES (Covariance Matrix Adaptation):

Particle Swarm Optimization (PSO)

Inspiration: Social behavior of bird flocking or fish schooling.

PSO Algorithm

For each particle i with position $\mathbf{x}_i$ and velocity $\mathbf{v}_i$:

1. Track personal best: $\mathbf{p}_i$ = best position particle i has visited
2. Track global best: $\mathbf{g}$ = best position any particle has visited
3. Update velocity:

$$\mathbf{v}_i \leftarrow \omega \mathbf{v}_i + c_1 r_1 (\mathbf{p}_i - \mathbf{x}_i) + c_2 r_2 (\mathbf{g} - \mathbf{x}_i)$$

where $r_1, r_2 \sim \text{Uniform}(0, 1)$

4. Update position:

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$$

Other Metaheuristics

Simulated Annealing:

- Probabilistic local search with controlled randomness
- Accepts worse solutions with probability $\exp(-\Delta f / T)$
- Temperature T decreases over time
- Guarantees convergence to global optimum (in limit)

Differential Evolution (DE):

- Create offspring by adding scaled difference of random individuals
- Simple and effective for continuous optimization
- Few hyperparameters

Ant Colony Optimization (ACO):

- Inspired by foraging behavior of ants
- Good for combinatorial problems (TSP, routing)

Tabu Search:

- Local search with memory to avoid cycling

Bayesian Optimization

Bayesian Optimization for Hyperparameter Tuning

Problem: Optimize expensive black-box function $f(\mathbf{x})$.

Examples:

- Hyperparameter tuning (training neural network expensive)
- A/B testing (each evaluation requires collecting data)
- Engineering design (simulations/experiments costly)

Bayesian Optimization

1. **Surrogate model:** Probabilistic model of f (typically Gaussian Process)

$$f(\mathbf{x}) \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$$

2. **Acquisition function:** Determines next point to evaluate

- Balance exploration vs exploitation
- Examples: Expected Improvement (EI), Upper Confidence Bound (UCB)

Gaussian Processes for Bayesian Optimization

Gaussian Process

Distribution over functions specified by:

- Mean function: $\mu(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$
- Covariance (kernel) function: $k(\mathbf{x}, \mathbf{x}') = \text{Cov}(f(\mathbf{x}), f(\mathbf{x}'))$

Common kernels:

- **RBF (Gaussian):** $k(\mathbf{x}, \mathbf{x}') = \sigma^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right)$
- **Matérn:** $k(\mathbf{x}, \mathbf{x}') = \sigma^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\sqrt{2\nu} \frac{r}{\ell}\right)^\nu K_\nu\left(\sqrt{2\nu} \frac{r}{\ell}\right)$

Posterior after observing data $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}$:

$$f(\mathbf{x})|\mathcal{D} \sim \mathcal{N}(\mu_{\mathcal{D}}(\mathbf{x}), \sigma_{\mathcal{D}}^2(\mathbf{x}))$$

Key property: Uncertainty quantification - GP provides both prediction and confidence.

Acquisition Functions

Expected Improvement (EI):

$$\text{EI}(\mathbf{x}) = \mathbb{E}[\max(0, f(\mathbf{x}) - f(\mathbf{x}^+))]$$

where $\mathbf{x}^+ = \arg \max_{\mathbf{x}_i \in \mathcal{D}} y_i$ is current best.

Closed form for GP:

$$\text{EI}(\mathbf{x}) = \begin{cases} (\mu(\mathbf{x}) - f(\mathbf{x}^+))\Phi(Z) + \sigma(\mathbf{x})\phi(Z) & \text{if } \sigma(\mathbf{x}) > 0 \\ 0 & \text{if } \sigma(\mathbf{x}) = 0 \end{cases}$$

where $Z = \frac{\mu(\mathbf{x}) - f(\mathbf{x}^+)}{\sigma(\mathbf{x})}$, Φ is standard normal CDF, ϕ is PDF.

Upper Confidence Bound (UCB):

$$\text{UCB}(\mathbf{x}) = \mu(\mathbf{x}) + \kappa\sigma(\mathbf{x})$$

where $\kappa > 0$ controls exploration-exploitation tradeoff.

Probability of Improvement (PI):

$$\text{PI}(\mathbf{x}) = \mathbb{P}(f(\mathbf{x}) > f(\mathbf{x}^+)) = \Phi(Z)$$

Practical Bayesian Optimization

Libraries:

- **scikit-optimize**: Simple BO for hyperparameter tuning
- **Optuna**: Modern, efficient hyperparameter optimization
- **Hyperopt**: Tree-structured Parzen estimator (TPE)
- **GPyOpt, BoTorch**: Advanced GP-based optimization
- **Ax/BoTorch (Facebook)**: Production-ready BO platform

Best practices:

- Use log scale for hyperparameters spanning orders of magnitude
- Parallel evaluations: batch Bayesian optimization
- Early stopping: prune unpromising trials
- Transfer learning: Use prior knowledge from related tasks

When to use Bayesian optimization:

- Function evaluations are expensive
- Derivatives not available
- Low to moderate dimensionality ($d < 20$)

Practical Optimization in Data Science

Choosing an Optimization Algorithm

Scenario	Method	Reason
Convex, smooth, small n	Newton/L-BFGS	Fast convergence
Convex, large n	SGD/Adam	Scalable
Deep learning	Adam/AdamW	Adaptive, robust
Non-smooth (e.g., Lasso)	Proximal gradient	Handles non-smoothness
Constrained, convex	Interior point	Polynomial time
Black-box, expensive	Bayesian opt.	Sample efficient
Non-convex, no gradients	CMA-ES, DE	Global search
Combinatorial	GA, ACO, tabu	Discrete optimization

Regularization in Optimization

Purpose: Prevent overfitting, improve generalization.

ℓ_2 regularization (Ridge):

$$\min_{\mathbf{w}} \mathcal{L}(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

- Shrinks weights toward zero
- Closed-form solution for linear models

ℓ_1 regularization (Lasso):

$$\min_{\mathbf{w}} \mathcal{L}(\mathbf{w}) + \lambda \|\mathbf{w}\|_1$$

- Induces sparsity (some weights exactly zero)
- Performs feature selection

Elastic Net:

$$\min_{\mathbf{w}} \mathcal{L}(\mathbf{w}) + \lambda_1 \|\mathbf{w}\|_1 + \frac{\lambda_2}{2} \|\mathbf{w}\|_2^2$$

Learning Rate Schedules

Constant: $\alpha_k = \alpha$

- Simple, but may not converge or converge slowly

Step decay: $\alpha_k = \alpha_0 \cdot \gamma^{\lfloor k/s \rfloor}$

- Reduce by factor γ every s epochs
- Common: $\gamma = 0.1$, $s = 30$ epochs

Exponential decay: $\alpha_k = \alpha_0 e^{-\lambda k}$

- Smooth decay

Cosine annealing: $\alpha_k = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min})(1 + \cos(\frac{k\pi}{K}))$

- Popular in deep learning
- Can restart periodically (SGDR)

Warm restarts: Periodically reset learning rate to high value

- Helps escape local minima

Common Issues

1. Loss not decreasing:

- Learning rate too high or too low
- Bug in gradient computation (use gradient checking!)
- Inappropriate initialization

2. Loss oscillating:

- Learning rate too high
- Reduce learning rate or use adaptive methods

3. Slow convergence:

- Learning rate too low
- Poor conditioning (try normalization, preconditioning)
- Use momentum or adaptive methods

Monitoring and Visualization

What to plot:

- **Loss curves:** Training and validation loss vs epochs/iterations
- **Gradient norms:** Check for vanishing/exploding gradients
- **Parameter norms:** Monitor weight magnitudes
- **Learning rate:** Track current learning rate
- **Evaluation metrics:** Accuracy, F1, etc.

Tools:

- **TensorBoard:** Comprehensive visualization for TensorFlow/PyTorch
- **Weights & Biases:** Experiment tracking and visualization
- **MLflow:** Model tracking and management
- **matplotlib/seaborn:** Custom plotting

Best practices:

- Log frequently but not excessively (every N iterations)
- Save checkpoints regularly
- Track hyperparameters with results

Example: Training a Model with PyTorch

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 # Define model, loss, optimizer
6 model = MyNeuralNetwork()
7 criterion = nn.CrossEntropyLoss()
8 optimizer = optim.Adam(model.parameters(), lr=0.001)
9 scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=100)
10
11 # Training loop
12 for epoch in range(num_epochs):
13     model.train()
14     total_loss = 0
15
16     for batch_idx, (data, target) in enumerate(train_loader):
17         # Forward pass
18         output = model(data)
19         loss = criterion(output, target)
20
21         # Backward pass and optimization
22         optimizer.zero_grad() # Clear gradients
23         loss.backward()        # Compute gradients
24
25         # Gradient clipping (optional, prevents exploding gradients)
26         torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
27
28         optimizer.step()        # Update parameters
29         total_loss += loss.item()
30
```

Advanced Topics

Multi-Objective Optimization

Problem: Optimize multiple conflicting objectives simultaneously.

$$\min_{\mathbf{x}} \{f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x})\}$$

Pareto Optimality

$\mathbf{x}^*$ is Pareto optimal if there exists no $\mathbf{x}$ such that:

- $f_i(\mathbf{x}) \leq f_i(\mathbf{x}^*)$ for all i
- $f_j(\mathbf{x}) < f_j(\mathbf{x}^*)$ for at least one j

The set of all Pareto optimal solutions forms the **Pareto front**.

Methods:

- **Scalarization:** $\min \sum_i w_i f_i(\mathbf{x})$ with weights w_i
- **ϵ -constraint:** Optimize one objective, constrain others
- **NSGA-II:** Non-dominated Sorting Genetic Algorithm

Distributed and Parallel Optimization

Data parallelism:

- Split data across workers
- Each worker computes gradient on its subset
- Aggregate gradients and update parameters

Model parallelism:

- Split model across devices (for very large models)
- Pipeline parallelism: different layers on different devices

Synchronous vs asynchronous:

- **Synchronous SGD:** Wait for all workers before updating
- **Asynchronous SGD:** Workers update independently (may be stale)

Communication efficiency:

- Gradient compression
- Local SGD: multiple local updates before synchronization
- AllReduce operations for efficient gradient aggregation

Federated Learning

Setting: Optimize model across decentralized data (e.g., mobile devices).

Federated Averaging (FedAvg)

Server:

1. Initialize global model $\mathbf{w}_0$
2. **For each round** $t = 1, 2, \dots$:
 - Select subset of clients
 - Send current $\mathbf{w}_t$ to selected clients
 - Receive local updates from clients
 - Average updates: $\mathbf{w}_{t+1} = \sum_k \frac{n_k}{n} \mathbf{w}_k^{(t+1)}$

Client k :

- Perform local SGD on local data for E epochs
- Send updated model to server

Neural Architecture Search (NAS)

Goal: Automatically find optimal neural network architecture.

Search space:

- Number of layers, layer types
- Number of neurons/filters
- Connections between layers
- Activation functions

Methods:

- **Reinforcement learning:** Train controller to generate architectures
- **Evolutionary algorithms:** Evolve architectures
- **Gradient-based:** DARTS (differentiable architecture search)
- **Bayesian optimization:** Model architecture performance

Efficiency techniques:

- Weight sharing across architectures
- Early stopping of poor architectures

Conclusion

Summary

Key Concepts:

- Convex vs non-convex optimization
- Gradient descent and variants (SGD, momentum, Adam)
- Constrained optimization (Lagrangian, KKT conditions)
- Evolutionary and metaheuristic algorithms
- Bayesian optimization for hyperparameter tuning
- Practical considerations (regularization, learning rates, debugging)

The Optimization Hierarchy

1. If convex and small: Use Newton/L-BFGS
2. If convex and large: Use (mini-batch) SGD with momentum/Adam
3. If non-convex with gradients: Use Adam/AdamW with proper initialization
4. If black-box, expensive: Use Bayesian optimization
5. If no gradients or discrete: Use evolutionary/metaheuristic algorithms

Best Practices

1. **Start simple:** Try Adam with default settings
2. **Normalize data:** Zero mean, unit variance
3. **Use validation set:** Monitor for overfitting
4. **Visualize:** Plot loss curves, gradients
5. **Gradient checking:** Verify implementation with finite differences
6. **Tune hyperparameters:** Learning rate most important
7. **Use regularization:** Weight decay, dropout, early stopping
8. **Be patient:** Deep models may need many epochs

Resources:

- Boyd & Vandenberghe: "Convex Optimization" (2004)

Future Directions

Emerging trends:

- **AutoML:** Automated machine learning pipelines
- **Neural ODEs:** Continuous-depth models, adjoint method
- **Meta-learning:** Learning to optimize, learned optimizers
- **Sparse optimization:** Efficient training of large models
- **Quantum optimization:** Quantum algorithms for optimization
- **Robust optimization:** Handling uncertainty and adversaries

Final Thought

Optimization is at the heart of machine learning and data science. Understanding optimization algorithms, their properties, and when to use them is essential for:

- Training models effectively
- Solving real-world problems
- Pushing the boundaries of what's possible with AI

Acknowledgments

- ESMAD for institutional support
- Mysense.ai for real-world optimization challenges
- The optimization research community
- Open-source contributors (PyTorch, TensorFlow, scikit-learn)

Generated with $\text{\LaTeX}$ Beamer

Theme: ESMAD Professional Academic Style

Diogo Ribeiro

ESMAD - Escola Superior de Média Arte e Design
Lead Data Scientist, Mysense.ai



dfr@esmad.ipp.pt



0009-0001-2022-7072



github.com/diogoribeiro7



linkedin.com/in/diogoribeiro7

This presentation is part of the academic materials repository.
Available at: <https://github.com/diogoribeiro7/academic-presentations>